Does the Four-Corner Code Know What a Character Looks Like?

Information content and similarity preservation
in the Four-corner encoding of Chinese characters

IB Mathematics: Applications and Interpretation HL
— Mathematical Exploration

Class of 2027

Student: Anthony Huang

Advisor: Mr. Peter Grimwood

1. Introduction

How would you look up a Chinese character in a dictionary? Most people would answer "Pinyin". But Pinyin requires that you know the phonetics of the character. What if you do not know its pronunciation? Today most people can handwrite it on a smartphone to look it up. Before smartphone, I learned a quieter system: the Four-Corner Method (四角号码) or the Four-Angle Number Method. You look at the four corners of a character, assign each corner a digit from 0 to 9 according to a fixed set of stroke-shape rules, and use the resulting four-digit code as an index to look up in a dictionary. For example, the character 林codes to 4499, also shown in Figure 1.

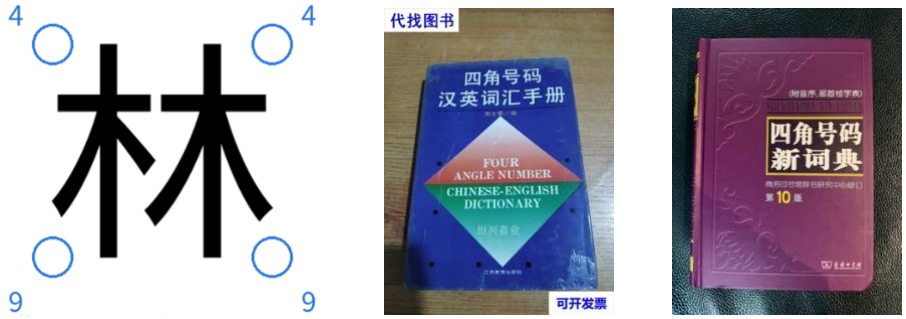


Figure 1. Left: the character 林 has the 4 corner code of 4499; Center: a second-handed Four-Angle Number dictionary; Right: my newly purchased Four-Corner Number dictionary.

1.1 Four-Corner Encoding Rules

Each character is assigned four digits, one per corner, in the order top-left, top-right, bottom-left, bottom-right. Generations of users memorized the classes through a four-line rhyme printed in the dictionaries themselves (Theobald):

横一垂二三點捺，叉四插五方框六。
七角八八九是小，點下有橫變零頭。
"Horizontal is 1, hanging is 2, and 3 stands for dots and slants; crosses 4, a stroke more 5, and boxes number 6. 7 corners, 8 like 八 'eight', and 9 like 小 'small'. Dot above a horizontal stroke is 0 in the front."

Figure 2 is a popular reference chart for the coding rules (Baidu Baike).



Figure 2. The ten Four-Corner stroke-shape classes (0–9), with worked character examples, as printed in a modern Chinese reference. Source: Baidu Baike ("Sijiao Haoma Chabiao").

The system was invented by Wang Yunwu in two steps: a preliminary "number indexing method," 「號碼檢字法」, originally connected to telegraph-code practice, in 1925 (Wang, "Haoma jianzifa"), refined the following year into the four-corner scheme proper, 《四角號碼檢字法》, in 1926 (Wang, "Sijiao Haoma Jianzifa"). It entered student dictionaries from 1928 and was later amended for simplified characters, before fading from everyday use as pinyin became universal (Liu). The digit-to-shape rules and corner order were summarized in English (Theobald).

The system has not disappeared entirely. It still used as a backup index in some dictionaries, as a way to keep one author's books together on Chinese library shelves, and as way to look up rare classical characters that pinyin-only indexes miss ("Four-corner method"; "Sijiao haoma"; "Sijiao Haoma Bu Neng Shao").

1.2 Personal Engagement

While I was at age of 10, I once flipped to a dictionary page headed 6010, finding several characters that share those corners, yet they looked nothing alike in the middle. That childhood impression is the inspiration of this exploration: if the code only records the corners, how much visual information does it keep? Mathematically, the method is interesting because it turns a printed character into four digits by hand as a lookup key, not as a visual similarity score. This creates one question with two connected dimensions: 1) do characters with nearby codes look alike? 2) how fully does the system use the 10^4 code space?

1.3 Research question

Research Question

To what extent does the Four-Corner Method use its available code capacity and preserve visual similarity among the 3,500 most common simplified Chinese characters, as measured by whole-code entropy and the relationship between Hamming distance and pixel overlap in Noto Sans SC Regular?

Hypothesis

Hamming distance (Hamming) and visual similarity do not have a strong correlation, given my personal experience of looking up characters in the dictionary. The capacity usage is probably over 90%, given I have seen all 10 digits in each corner of a character.

The investigation aims to answer this question through 2 dimensions:

- **Similarity preservation:** the relationship between Hamming distance (the number of digit places where two codes disagree) and visual similarity. I assess the relationship using Spearman r_s
- **Capacity use**: the proportion of the theoretical capacity of the information content of 10^4 equally likely codes used, measured using whole-code entropy (the average information a real code carries), and the sources of unused capacity.

1.4 Exploration Methodology

1.4.1 Similarity-preservation dimension

Hamming distance

Hamming distance is simply "how many of the four-digit places disagree?" For codes $x = (x_1, x_2, x_3, x_4)$ and $y = (y_1, y_2, y_3, y_4)$,

$$d_H(x, y) = \sum_{j=1}^{4} 1\left(x_j \neq y_j\right) \in \{0, 1, 2, 3, 4\}$$

Here $1(x_j \neq y_j)$ is 1 when condition $x_j \neq y_j$ and 0 otherwise — so each corner that differs adds 1 to the total. For example, 6010 and 6021 agree on the first two digits, so $d_H = 2$. A code compared with itself has $d_H = 0$; codes that differ in every place (e.g. 6010 and 4499) have $d_H = 4$.

Table 1 list a few more examples. Digit values do not matter: 1 vs 2 counts the same as 1 vs 9.

Pair	Codes	d_H
日 / 旦	6010 / 6010	0
国 / 园	6010 / 6021	2
国 / 日	6010 / 6010	0
国 / 林	6010 / 4499	4

Table 1. Example Hamming distances between codes in the dataset.

Visual similarity

Every character we see on a screen is ultimately rendered as a bitmap, a grid of pixels. Figure 3 shows ten characters in 64x64 bitmap, with ink fractions, defined as # of black pixels/64x64.

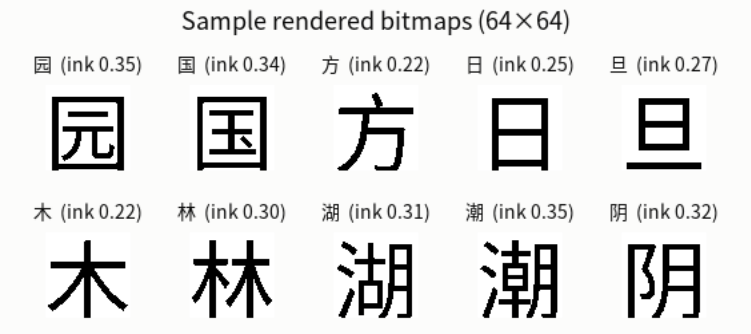


Figure 3. 10 characters rendered at 64x64, in font Noto Sans SC Regular**,** labelled with ink fractions.

In this investigation, visual similarity means Jaccard pixel overlap S: the proportion of black pixels shared by two characters among all black pixels belonging to either character, after both are rendered in fixed font Noto Sans SC Regular and binary image frame. Take two 64×64 black-and-white drawings: Let A be the set of black pixels of the first and B of the second. The score is the shared black pixels divided by all black pixels that appear in either picture:

$$S(A, B) = \frac{|\, A \cap B \,|}{|\, A \cup B \,|} \in [0, 1]$$

The study will measure the similarity-preservation dimension using two quantities fixed before looking at the main results:

1. Overall link: among 2,000 randomly chosen character pairs from all $\binom{3500}{2} = 6{,}123{,}250$ possible pairs, how strongly does d_H go with visual similarity? This will be summarized by Spearman rank (Spearman) correlation r_s .
2. Close vs far: among samples built so every distance $0,1,2,3,4$ is well represented, how much higher is median similarity when codes match ($d_H = 0$) than when they differ in all four places ($d_H = 4$)? That gap is formally expressed as $\Delta = \tilde{S}_0 - \tilde{S}_4$.

1.4.2 Capacity-use dimension

A four-digit code with digits 0-9 has $10^4 = 10{,}000$ possible values. Following Shannon's definition of entropy (Shannon), the information content, in bits, of a character's corner position j is

$$H_j = -\sum_{i=0}^{9} p_j(i)\log_2 p_j(i)\,,\ 0 \le H_j \le \log_2 10 \approx 3.3219, 1 \le j \le 4.$$

It is highest when characters are spread evenly across the available codes and lower when many characters are concentrated in the same codes: $0 \le H_j \le \log_2 10 \approx 3.3219$ (Shannon). Thus, the full space of 4 digits for a character is $4\log_2 10 \approx 13.29$ bits. The $H(C)$ for the 3,500 characters will be calculated, compared with the theoretical maximum, the unused capacity decomposed into predictable digits and dependence between corners.

1.4.3 Computational Procedures

The exploration uses Python 3 programs with Pillow, NumPy, and Matplotlib to help automate some of the tasks, including:

- Rendering the characters
- Calculating two characters' pixel intersection, union and S score
- Calculation of d_H, digit and code frequencies, entropy, spearman correlations, intervals, permutation, etc
- Plotting of graphs.

2 Data Collection

2.1 Raw Data Collection

The most common 3,500 characters were first picked from the Jun Da frequency list: https://lingua.mtsu.edu/chinese-computing/statistics/char/list.php?Which=MO

Next, each character's unicode (for identification purpose) was printed and recorded using this line of python code: `print(character, f"U+{ord(character):04X}")`

Then, each of these characters' 4-corner codes were looked up manually, using the new dictionary in Figure 1. Since the Summer, a sustained effort of daily manual lookup and verification of 100 characters' codes have ultimately led to the completion of the raw data collection table in Table 2.

char	unicode	rank	code4	d1	d2	d3	d4
的	U+7684	1	2762	2	7	6	2
一	U+4E00	2	1000	1	0	0	0
国	U+56FD	20	6010	6	0	1	0
…	…	…	…	…	…	…	…
林	U+6797	364	4499	4	4	9	9
…	…	…	…	…	…	…	…
锚	U+951A	3500	8476	8	4	7	6

Table 2. Sample rows from the **raw data collection** of the top 3,500 most common Chinese characters, with hand-verified corner readings from the dictionary. d1–d4 are the four corner digits that make up code4, one per corner in reading order TL (top-left) · TR (top-right) · BL (bottom-left) · BR (bottom-right).

2.2 Overall Distribution of d_H

There are $\binom{3500}{2} = 6{,}123{,}250$ unordered pairs. Their Hamming distribution is as below:

d_H	0	1	2	3	4
Number of pairs	5,929	83,932	560,962	2,027,467	3,444,960
Share	0.1%	1.4%	9.2%	33.1%	56.3%

Table 3. Hamming-distance distribution over all $\binom{3500}{2}$ character pairs.

2.3 Simple Random Sample (SRS) and Stratified Median S Scores

2.3.1 SRS

A simple random sample of 2000 pairs, seed 2026 and without replacement, from the 3,500-character universe.

A / B	CODES	d_H	S
了 / 诈	1720 / 3871	4	0.0841
人 / 婚	8000 / 4246	4	0.1871
人 / 栖	8000 / 4196	4	0.1269
人 / 杏	8000 / 4060	2	0.1731
有 / 婢	4022 / 4644	3	0.2683

⋮	⋮	⋮	⋮ (1,995 more rows)

Table 4. The first five of the 2,000 SRS pairs (seed 2026); the rest omitted for space.

Spearman correlation of midranks r_i (for d_H) and s_i (for S): ("scipy.stats.pearsonr.")

$$r_s = \frac{\sum_i (r_i - \bar{r})(s_i - \bar{s})}{\sqrt{\sum_i (r_i - \bar{r})^2 \sum_i (s_i - \bar{s})^2}}.$$

- r_i, s_i — midranks of d_H and S for pair i.
- $\bar{r}, \bar{s}$ — mean of each midrank list.

A Spearman Correlation Fitting of Table 4 data gives r_s = -0.0501.

2.3.2 Similarity vs Hamming Stratum

A stratified sample, up to **500** pairs without replacement, at each $d_H = 0,1,2,3,4$, seed 2027**.**

d_H	A / B	Codes	S
0	面 / 吾	1060 / 1060	0.2806
1	抗 / 尧	5001 / 5021	0.1673
2	柏 / 董	4690 / 4410	0.3146
3	柏 / 嫦	4690 / 4942	0.3294
4	公 / 猜	8073 / 4522	0.1729
⋮	⋮	⋮	⋮ (499 more rows per stratum)

Table 5. One example pair from each of the 5 Hamming-distance strata; 499 more per stratum not shown.

2.3.3 Summary

Quantity	Value	Sample
Spearman $\mathbf{r_s}$ (midranks)	−0.050	SRS, n = 2000
Median **S** at $\mathbf{d_H = 0, 1, 2, 3, 4}$	0.333 · 0.297 · 0.274 · 0.255 · 0.257	stratified, 500/stratum
$\mathbf{\Delta = \tilde{S}_0 - \tilde{S}_4}$	+0.075	stratified

Table 6. Result summary of SRS and Stratum analysis. Spearman $\mathbf{r_s}$ measures the rank correlation between d_H and S of the 2000 random pairs in Table 4.

2.4 Human Ratings vs Similarity

Item	Char_a	Char_b	Rating_1	Rating_2	d_H	S
1	呀	吁	4	5	0	0.5543
2	正	五	3	3	0	0.3626
3	烛	佃	3	2	4	0.3328
…	…	…	…	…	…	…
40	限	氛	1	1	3	0.2729

Table 7. Human Ratings form data (red columns not shown to the raters).

3 Analysis & Discussion

1. There is a weak correlation between the ranking orders of d_H and S.

The 2,000 random pairs in Table 4 from the 3,500 characters universe were plotted in Figure 4. Their $\mathbf{d_H}$ and S are converted to midranks and combined by the formula in section 2.3.1, giving $r_s = -0.0501$, a weak negative correlation.

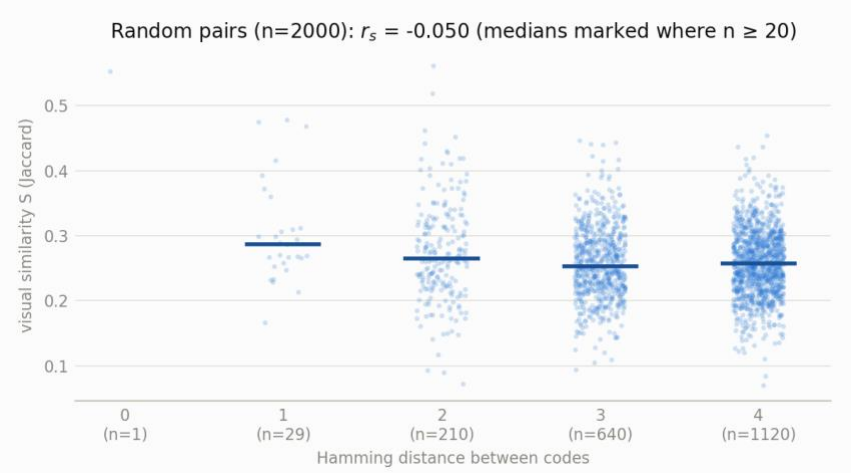


Figure 4. Plotting the 2,000 simple random pairs from the 3,500-character universe in Table 4. Horizontal jitter is added for visibility; median bars are shown only for distances with at least 20 pairs.

So, Is $r_s = -0.0501$ just noise, or a real (if tiny) link? I chose a two-tailed t-test to check this.

Let ρ_s be the population Spearman rank correlation between $\mathbf{d_H}$ and S.

$$H_0: \rho_s = 0 \text{ (no population association)}, \; H_1: \rho_s \neq 0$$

Two-sided H_1 matches the similarity-preservation dimension of the research question ("is there a link?"). Since this is as correlation t-test, the degrees of freedom $\mathrm{df} = \mathrm{n} - 2 = 1998$. The test statistic under H_0 is (Zar):

$$\mathrm{t} = \mathrm{r_s}\sqrt{\frac{\mathrm{n}-2}{1-\mathrm{r_s^2}}},\ \mathrm{df} = \mathrm{n} - 2$$

For $\mathrm{r_s} = -0.0501$ and $\mathrm{n} = 2000$,

$$\mathrm{t} = -0.0501\sqrt{\frac{1998}{1-0.0501^2}} \approx -2.24,\ \mathrm{p} \approx 0.025$$

The standard normal table gives $A(2.24) = 0.9875$, the area to the left of 2.24 (Dougherty, table A.1), so the tail beyond $+2.24$ holds $1 - 0.9875 = 0.0125$; adding its mirror image below -2.24 gives $\mathrm{p} \approx 2 * (0.0125) = 0.025$. Since $0.025 < 0.05$, chance alone is an unlikely explanation, so I reject H_0 at the 5% level. This means there is a correlation link between the visual similarity S and the hamming distance $\mathrm{d_H}$, but the link is weak since $\mathrm{r_s}$ is small.

2. Stratum analysis reveals that Hamming distance carries some information about visual similarity, but individual pairs vary substantially

Next, 500 random pairs were chosen from for groups of $\mathrm{d_H} = 0,1,2,3,4$ of the 3,500-character universe, respectively. Figure 5 plots the boxpots of the data in Table 5, S vs $\mathrm{d_H}$. The median falls from 0.333 at $d_H = 0$ to about 0.255–0.257 at $d_H = 3$–4, but the groups overlap substantially. Same-code pairs ($d_H = 0$) are a bit more similar on average than fully different-code pairs, so the corners seem to keep some shape signal.

The difference in visual similarity of two groups, $\mathrm{d_H} = 0$ and $\mathrm{d_H} = 4$, is computed as:

$$\Delta = \tilde{\mathrm{S}}_0 - \tilde{\mathrm{S}}_4 \cong 0.075.$$

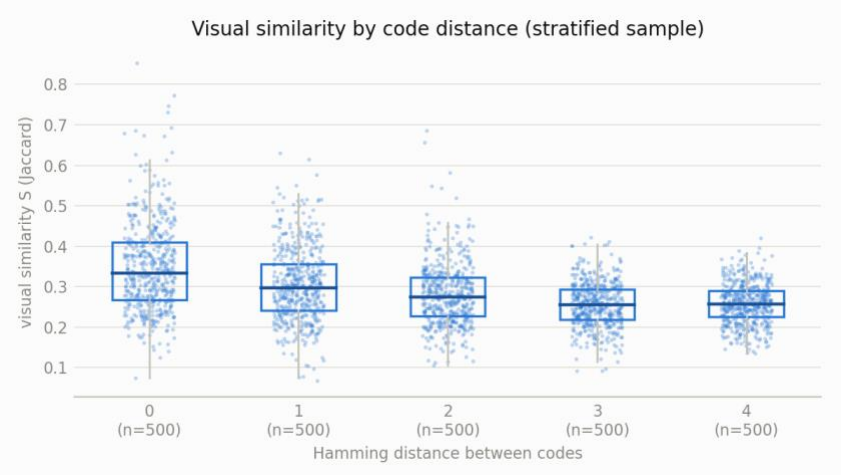


Figure 5. Boxplots of pixel-overlap similarity by Hamming distance, using a stratified sample. The five source pools contain 5,929, 83,932, 560,962, 2,027,467, and 3,444,960 pairs, with median S valule of $0.333 \cdot 0.297 \cdot 0.274 \cdot 0.255 \cdot 0.257$ for $d_H = 0,1,2,3,4$, respectively; 500 pairs were randomly selected without replacement from each pool.

I then looked into each stratum's character pairs that have the the least, nearest-to-median and the largest S values and plot them in Figure 6:

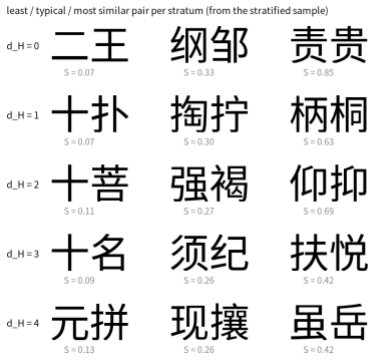


Figure 6. For each Hamming stratum, the least similar, nearest-to-median, and most similar pairs.

Figure 6 shows that within each Hamming distance stratum, the similarity scores have a large range, consistent with our finding that Hamming distance carries some information about visual similarity, but individual pairs vary substantially.

3. Jaccard similarity is a limited proxy for human judgement

To check how the visual similarity metric tracks what humans see, 40 random pairs (8 from each d_H group, seed 2028) were drawn from the stratified sample, shuffled, and shown without codes or S scores to two human raters. See Table 7. The 2 raters saw ordinary

smooth printed characters (72 px). Two raters scored each pair from 1 (no resemblance) to 5 (nearly identical) on printed shape only.

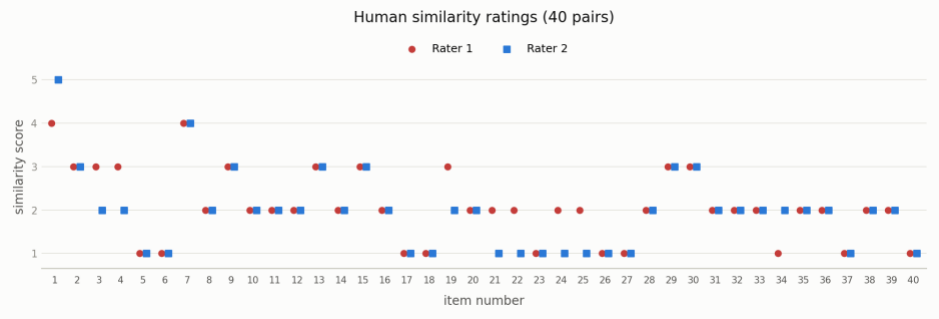


Figure 7. Both raters' raw similarity scores for all 40 character pairs in one plot.

Each rater's 40 scores are ranked separately, with ties assigned average ranks, called midranks, and the two sets of ranks matched by character pair. Spearman correlation of these midranks gives $r_s = 0.835$, indicating strong consistency in how the raters order the pairs by similarity.

The data in Table 7 is plotted in Figure 8, mean of the two ratings vs similarity. The plot shows a positive relationship between the average human similarity rating and Jaccard pixel overlap across the 40 pairs with $r_s = 0.269$. The considerable scattering indicates that Jaccard pixel overlap does not closely track human judgements, so it is a limited proxy for human judgement.

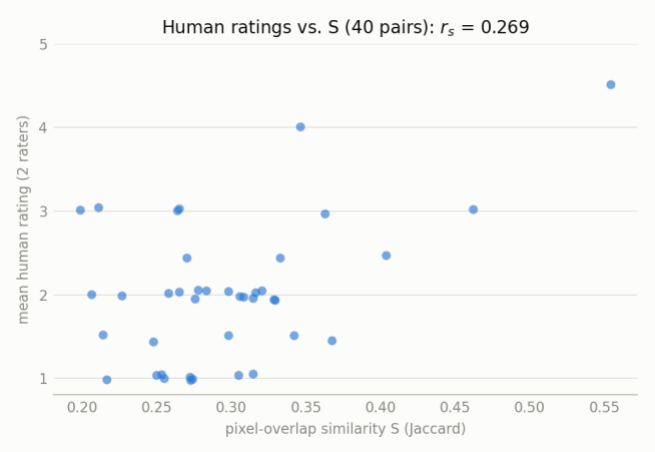


Figure 8. Mean of two human ratings versus Jaccard similarity. Data: 40 pairs, seed 2028.\

4. The digits at each corner collectively carry different amount of information (H)

For each corner $j = 1,2,3,4$, let $p_j(i)$ be the fraction of the 3,500 characters whose jth digit is i ($i = 0, \ldots, 9$), per Shannon's definition of entropy (Shannon 393), the information content of position j is $H_j = -\sum_{i=0}^{9} p_j(i)\log_2 p_j(i)$ (bits), $0 \le H_j \le \log_2 10 \approx 3.3219$.

At position 1 (top-left), 175 of the 3,500 characters have digit 0, so $p_1(0) = 175/3500 = 0.0500$, and this one digit contributes $-p_1(0)\log_2 p_1(0) = -0.05 \times \log_2 0.05 \approx -0.05 \times (-4.3219) \approx$ 0.2161 bits to the sum. In Table 8, each table cell has the form **count · $p_j(i)$ · $-p_j(i)\log_2 p_j(i)$**:

Digit i	Position 1 (TL)	Position 2 (TR)	Position 3 (BL)	Position 4 (BR)
0	175 · 0.0500 · 0.2161	859 · 0.2454 · 0.4974	509 · 0.1454 · 0.4045	844 · 0.2411 · 0.4948
1	309 · 0.0883 · 0.3091	312 · 0.0891 · 0.3109	679 · 0.1940 · 0.4590	615 · 0.1757 · 0.4408
2	586 · 0.1674 · 0.4317	278 · 0.0794 · 0.2902	800 · 0.2286 · 0.4867	658 · 0.1880 · 0.4533
3	529 · 0.1511 · 0.4120	219 · 0.0626 · 0.2502	182 · 0.0520 · 0.2218	400 · 0.1143 · 0.3576
4	589 · 0.1683 · 0.4327	456 · 0.1303 · 0.3831	284 · 0.0811 · 0.2940	355 · 0.1014 · 0.3349
5	299 · 0.0854 · 0.3032	162 · 0.0463 · 0.2052	95 · 0.0271 · 0.1412	109 · 0.0311 · 0.1559
6	320 · 0.0914 · 0.3155	194 · 0.0554 · 0.2313	212 · 0.0606 · 0.2450	205 · 0.0586 · 0.2398
7	314 · 0.0897 · 0.3121	683 · 0.1951 · 0.4600	331 · 0.0946 · 0.3218	58 · 0.0166 · 0.0980
8	220 · 0.0629 · 0.2509	270 · 0.0771 · 0.2851	154 · 0.0440 · 0.1983	160 · 0.0457 · 0.2035
9	159 · 0.0454 · 0.2026	67 · 0.0191 · 0.1092	254 · 0.0726 · 0.2746	96 · 0.0274 · 0.1423
Sum	3,500 · 1.0000 · **3.1860**	3,500 · 1.0000 · **3.0227**	3,500 · 1.0000 · **3.0470**	3,500 · 1.0000 · **2.9209**

Table 8. All ten terms of H_j at each corner, computed from the 3,500 character universe. Within each position's column, the ten counts sum to 3,500 and the ten proportions sum to 1, as they must; the ten bit-contributions give $H_1, \ldots, H_4 = 3.1860, 3.0227, 3.0470, 2.9209$ bits.

As discussed in section 1.4.2, H_j is highest when characters are spread evenly across the available digits [0,1,2,3,4,5,6,7,8,9], when $H_j = \log_2 10 \approx 3.3219$ (Shannon). So clearly our data in Table 8 shows the characters are not using the digits evenly as H_1, H_2, H_3, H_4 are all less.

Figure 9 is a plot of the digit distribution at the 4 corners of the 3,500 characters.

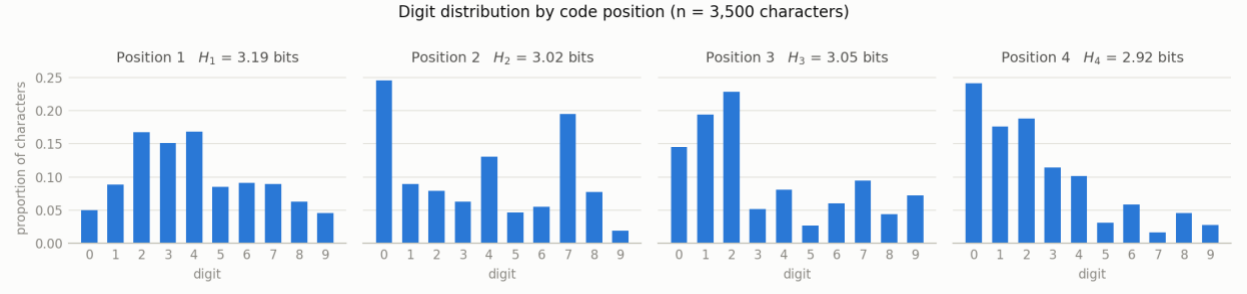


Figure 9. Empirical digit distributions at the four corners of the 3500 characters.

5. Not all digits are even, and corners are not independent

Some corners of a character can be more fixed together due to certain radicals often span adjacent corners (See Figure 6's second column). If the top-left digit often forces what the bottom-left digit is (e.g. both fixed by the same left radical), the second digit is partly redundant.

So I borrowed the concept of *Mutual information* to measure how much two corners overlap in that way. Let X_j and X_k be the digits at corners j and k across the 3,500 characters, the **mutual information** $I(X_j; X_k)$ measures two variables' shared information (Cover and Thomas 21):

$$I(X_j; X_k) = H_j + H_k - H(X_j, X_k)$$
$$= \sum_{a,b} p_{jk}(a,b) \log_2 \frac{p_{jk}(a,b)}{p_j(a)\, p_k(b)} \geq 0$$

- $I(X_j; X_k)$ (bits) — the mutual information itself: how much knowing one corner's digit reduces uncertainty about the other.
- H_j, H_k — how unpredictable each corner is on its own
- $H(X_j, X_k)$ — entropy of the *pair* of digits treated as one combined outcome (10x10 = 100 possible pairs)
- $p_{jk}(a,b)$ — fraction of characters with digit a at corner j and b at corner k
- $p_j(a)$, $p_k(b)$ — single-corner frequencies (row/column totals of that table)

$I = 0$ exactly when the corners are independent (the joint table is just the product of the margins). Large I means the two corners carry partially overlapped information. Table 9 shows how the mutual information of corners 1 and 3: $I(X_1; X_3)$ is calculated:

a (pos. 1)	b (pos. 3)	O_{ab}	$p_{13}(a,b)$	$p_1(a)\, p_3(b)$	term (bits)
5	0	199	0.0569	0.0124	0.1248
6	0	189	0.0540	0.0133	0.1092
7	2	190	0.0543	0.0205	0.0762

3	1	214	0.0611	0.0293	0.0648
4	9	128	0.0366	0.0122	0.0579
2	2	227	0.0649	0.0383	0.0494
⋮	⋮	⋮	⋮	⋮	⋮
Sum (100 rows)		3,500	1.0000	1.0000	**0.5275**

Table 9. Calculating mutual information of corners 1 and 3: $I(X_1; X_3)$.
Each *term* is $p_{13}(a,b)log_2\left(p_{13}(a,b)/\left(p_1(a)p_3(b)\right)\right)$, with $p_1(a)$ and $p_3(b)$ tabulated in Table 8.

All six pairs of the corners are computed similarly and reported in Table 10:

Pair	1–2	1–3	1–4	2–3	2–4	3–4
I	0.18	**0.53**	0.03	0.03	**0.14**	**0.19**

Table 10. Pairwise mutual information (bits) between the four corner positions.

Table 10 shows that the strongest dependence is between corner positions 1 and 3 - the two **left** corners. This makes sense because a left-side radical such as 氵 (water), 木 (wood), or 讠 (speech) occupies both left corners at once, so those digits are partly redundant. The water-radical in 湖、洞、潮、润 is a concrete example: shared left radical locks the left digits together across many characters. By raw I value, the next-highest pair is actually corners 3 & 4 (I = 0.19 bits) – the **bottom** corners, likely due to the bottom-side radicals like the one in 思、想、忘 or 热、煮、点 ; corners 2 & 4 -- the two **right** corners, likely due to the right-sided radicals such as the one in 利、到、别 or the one in 都、部、郊. Figure 10 shows the dependency between corner positions with a colormap.



Figure 10. For each of the six corner-position pairs, the joint digit distribution independence would predict from that pair's separate digit popularities, $p_j(a)\,p_k(b)$ (left subplot), next to the observed joint distribution (right subplot). light = rare, dark = common — so like is compared with like.

6. There are many code collisions and information capacity-usage is inefficient

Out of the 3,500 characters, there are a total of 1,578 distinct 4-corner codes, so there are collisions among them.

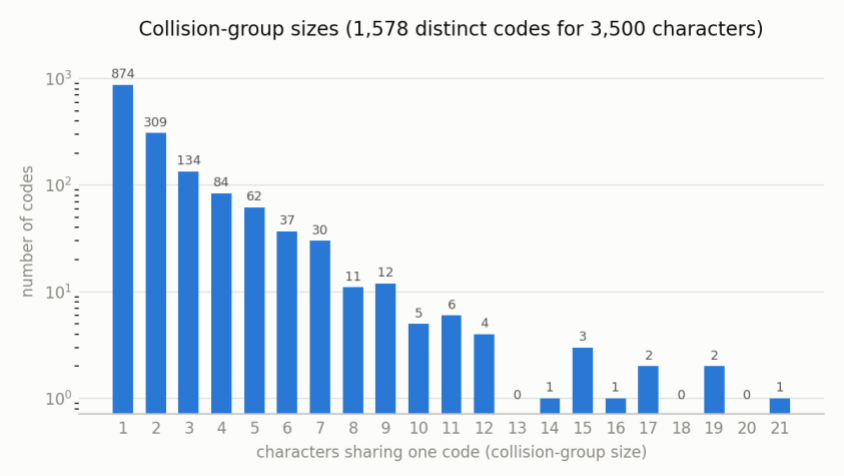


Figure 11. Distribution of collision-group sizes (logarithmic Y-axis; each bar labelled with exact count)

If we treat the whole four-digit code as one label: C stands for the complete code (X_1, X_2, X_3, X_4) of a randomly picked character, 6010 for 国, 4499 for 林, so C is a single code, but a *random* one. Which code we get depends on which character was picked, just as we roll a dice. How unpredictable is that draw? That is $H(C)$, the same entropy formula applied to whole codes instead of single digits. C could in principle take any of the $10^4 = 10{,}000$ possible values, but only 1,578 appear; $H(C)$ is the entropy of how those codes are shared:

$$H(C) = -\sum_c q_c \log_2 q_c,$$

$$q_c = \frac{\text{number of characters with code } c}{3500}.$$

q_c is *per distinct code* — 1,578 of them, from $q_{0022} = 21/3500$ (方、高、市、商、离、育、帝、序、席、旁、齐、腐、鹰、廊、庸、斋、彦、膏、裔、廖、廓) for the biggest group down to $1/3500$ for a code carried by a single character, and together they sum to 1.

For example, the whole code 6010 is shared by 11 of the 3,500 characters, so $q_{6010} = 11/3500 = 0.003143$, contributing

$$-q_{6010}\log_2 q_{6010} = -0.003143 \times \log_2 0.003143 \approx -0.003143 \times (-8.3145) \approx \mathbf{0.0261} \text{ bits}$$

to the sum. The same calculation is made for every distinct code:

Code c	Characters with this code	q_c	$-q_c\log_2 q_c$ (bits)
0022	21	0.006000	0.0443
7722	19	0.005429	0.0409
4460	17	0.004857	0.0373
6010	11	**0.003143**	**0.0261**
3113	1	0.000286	0.0034
⋮	⋮	⋮	⋮
Total over all 1,578 codes	3,500	1.000000	**10.1534**

Table 11. The whole-code entropy calculation. The final row is the sum over all 1,578 codes.

For this investigation, let us calculate the **capacity-use ratio** as the whole-code entropy divided by the largest possible entropy, $4\log_2 10 = 13.2877$, when characters use all digits at each corner evenly (section 1.4.2):

$$\frac{H(C)}{4\log_2 10} = \frac{10.1534}{13.2877} = 0.7641 = 76.4\%.$$

So, the real codes use about 76.4% of the full 13.2877-bit room. To find out where the rest is lost, first add the four positional entropies from Table 8:

$$\sum_{j=1}^{4} H_j = 3.1860 + 3.0227 + 3.0470 + 2.9209 = 12.1766 \text{ bits.}$$

Both $\sum H_j$ and $H(C)$ use the same 3,500-character universe. Each H_j measures uncertainty about one corner's digit without using knowledge of the other corners. It already accounts for some digits being more common than others. In contrast, $H(C)$ measures uncertainty about the complete four-digit code, using the frequencies of the combinations observed.

To isolate dependence from unequal digit frequencies, keep each corner's individual digit frequencies fixed. If the four corners were independent, knowing any of the other corners would

provide no information about a corner's digit. Each corner would therefore contribute its full uncertainty, and the whole-code entropy would equal $\sum \mathrm{H_j} = \mathrm{H_1} + \mathrm{H_2} + \mathrm{H_3} + \mathrm{H_4}$ (Watanabe 66).

When the corners are dependent, knowing some corners reduces the uncertainty about the remaining corners, on average. The actual whole-code entropy $\mathrm{H(C)}$ is therefore lower than this independent-corner baseline. The difference $\sum \mathrm{H_j} - \mathrm{H(C)}$ measures that reduction. It compares distributions with the same individual digit frequencies, so it separates dependence from the effect of unequal digit frequencies.

For the full set of 3,500 characters, the total unused capacity can thus be split into two parts:

$$13.2877 - \mathrm{H(C)} = \left(13.2877 - \sum_{\mathrm{j}=1}^{4} \mathrm{H_j}\right) + \left(\sum_{\mathrm{j}=1}^{4} \mathrm{H_j} - \mathrm{H(C)}\right).$$

The $-\sum \mathrm{H_j}$ and $+\sum \mathrm{H_j}$ cancel. The first part compares each corner's actual entropy with the maximum it could have if its ten digits were used equally:

$$\begin{aligned} 13.2877 - \sum \mathrm{H_j} &= 13.2877 - 12.1766 \\ &= 1.1111 \approx 1.11 \text{ bits}, \\ \frac{1.1111}{13.2877} \times 100\% &\approx 8.4\%. \end{aligned}$$

The second part is Watanabe's **total correlation** (Watanabe 66). It measures the further reduction from the independent-corner baseline to the actual whole-code entropy, after unequal digit frequencies have already been accounted for:

$$\begin{aligned} \mathrm{T} = \sum \mathrm{H_j} - \mathrm{H(C)} &= 12.1766 - 10.1534 \\ &\approx 2.02 \text{ bits}, \\ \frac{2.0231}{13.2877} \times 100\% &\approx 15.2\%. \end{aligned}$$

Total correlation is zero exactly when the four corner positions are mutually independent. Here, the approximately 2.02-bit difference measures dependence within the selected character population.

Table 12 lists the final breakdown of the theoretical information capacity:

Component	Bits	Share of 13.29
Theoretical capacity $4\log_2 10$	13.29	100.0%
Used: $H(C)$	10.15	76.4%
Lost to non-uniform digits: $13.29 - \sum H_j$	1.11	8.4%
Lost to corner dependence: $T = \sum H_j - H(C)$	2.02	15.2%

Table 12. Capacity decomposition of the four-digit code into used, non-uniformity-lost, and dependence-lost bits.

4 Conclusion

This exploration investigated to what extent the Four-Corner Method uses its available code capacity and preserves visual similarity among the 3,500 most common simplified Chinese characters. By applying Hamming distance, Jaccard pixel similarity, Spearman rank correlation, Shannon entropy, and mutual information, I found that the Four-Corner Method preserves some information on character shapes but does not use its theoretical information capacity efficiently.

For the similarity-preservation dimension, the Spearman correlation between Hamming distance and Jaccard similarity shows a very weak negative relationship ($r_s = -0.0501$). Although this relationship was significant at the 5% level, it was too weak to make Hamming distance a reliable predictor of visual similarity. The stratified analysis supported this interpretation: the median Jaccard similarity decreased from 0.333 for pairs with $d_H = 0$ to 0.257 for pairs with $d_H = 4$, a difference of 0.075. However, two characters with identical codes may still look very different, while characters with entirely different codes can appear similar. Therefore, the Four-Corner Method retains a limited amount of visual information, and code similarity does not strongly predict whole-character similarity.

For the capacity-use dimension, the whole-code entropy was 10.15 bits, compared with the theoretical maximum of 13.29 bits. This corresponds to approximately 76.4% of the theoretical capacity, lower than my initial hypothesis of over 90%. The decomposition of the remaining capacity revealed that approximately 1.11 bits (8.4%) were lost due to unequal digit frequencies, with a further 2.02 bits (15.2%) lost because of dependence between corner positions. The

mutual information analysis also showed that the strongest pairwise dependence occurred between the top-left and bottom-left corners, with approximately 0.53 bits of shared information. This is consistent with the presence of radicals that occupy both left corners of Chinese characters.

These results helped me reinterpret my childhood observation of finding visually different characters under the same dictionary code. Initially, I suspected that the Four-Corner Method preserved no visual similarity, while using most of its available code space. The mathematical analysis revealed a more complex picture: the method does retain a small amount of shape information, but its capacity is considerably reduced by the structural characteristics of Chinese characters.

5 Limitations

This investigation has several limitations:

First, Jaccard pixel overlap may not accurately represent human perception of visual similarity. The weak correlation ($r_s = 0.269$) between Jaccard similarity and human ratings also suggests that pixel overlap is an imperfect measure. Using different fonts or structural similarity metrics could improve the validity of the comparison.

Second, the dataset was limited to the 3,500 most common simplified Chinese characters, which may not represent the entire Chinese writing system well. Including rare and traditional characters could help reveal different code distributions and dependencies.

Third, the similarity analysis relied on sampled character pairs. The 2,000 randomly selected pairs may not fully represent the more than six million possible pairs. Repeating the sampling or analyzing all possible pairs would improve the reliability of the conclusions.

Despite these limitations, the investigation provides useful mathematical evidence that the Four-Corner Method preserves limited visual similarity and uses only part of its information capacity.

References

Cover, Thomas M., and Joy A. Thomas. Elements of Information Theory. John Wiley & Sons, 1991.

Da, Jun. “Modern Chinese Character Frequency List.” Chinese Text Computing, Middle Tennessee State University, 2004, https://lingua.mtsu.edu/chinese-computing/statistics/char/list.php?Which=MO.

Dougherty, C. Statistical Tables. 2002, tables A.1–A.2, computed to accompany Introduction to Econometrics, 2nd ed., Oxford University Press, 2002, https://home.ubalt.edu/ntsbarsh/business-stat/StatistialTables.pdf.

“Four-corner method.” Wikipedia, Wikimedia Foundation, 29 June 2026, https://en.wikipedia.org/wiki/Four-corner_method.

Hamming, R. W. “Error Detecting and Error Correcting Codes.” The Bell System Technical Journal, vol. 29, no. 2, Apr. 1950, pp. 147–160.

Liu Ying 劉螢. “Wang Yunwu yu sijiao haoma jianzifa” 王雲五與四角號碼檢字法 [Wang Yunwu and the Four-Corner Indexing Method]. Shumu jikan 書目季刊 [Bibliography Quarterly], vol. 36, no. 1, June 2002, pp. 21–42, https://doi.org/10.6203/BQ.2002.6.36.1.02.

“Noto Sans Simplified Chinese.” Google Fonts, Google, https://fonts.google.com/noto/specimen/Noto+Sans+SC.

“scipy.stats.pearsonr.” SciPy Documentation, SciPy Developers, https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.pearsonr.html.

“scipy.spatial.distance.jaccard.” SciPy Documentation, SciPy Developers, https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.distance.jaccard.html.

Shannon, C. E. “A Mathematical Theory of Communication.” The Bell System Technical Journal, vol. 27, no. 3, July 1948, pp. 379–423.

“Sijiao haoma” 四角号码 [Four-Corner Code]. Wikipedia, Wikimedia Foundation, 25 Apr. 2026, https://zh.wikipedia.org/wiki/四角号码.

“Sijiao Haoma Bu Neng Shao” 四角号码不能少 [Four-Corner Code Should Not Be Omitted]. Nanjing University Campus Newspaper 南京大学校报, no. 922, 20 June 2006, https://xiaobao.nju.edu.cn/d6/03/c18184a316931/page.htm.

“Sijiao Haoma Chabiao” 四角号码查字表 [Four-Corner Code Lookup Table]. Baidu Baike, Baidu, https://bkimg.cdn.bcebos.com/pic/5beeba0f277b8ad4ab64579e.

Sijiao Haoma Xin Cidian 四角号码新词典 [Four-Corner Code New Dictionary]. Photograph, online secondhand-book listing, https://t13.baidu.com/it/u=746524076,353134274&fm=224&app=112&f=JPEG?w=500&h=500.

Spearman, C. “The Proof and Measurement of Association between Two Things.” The American Journal of Psychology, vol. 15, no. 1, 1904, pp. 72–101.

Tao Shihao 陶士豪, editor. Sijiao Haoma Han-Ying Cihui Shouce 四角号码汉英词汇手册 [Four Angle Number Chinese-English Vocabulary Handbook]. Jiangxi Education Press. Photograph, online marketplace listing, https://img14.360buyimg.com/pop/jfs/t1/408897/25/10617/241626/69c6281dFff4d5309/00836408555d95d6.jpg.

Theobald, Ulrich. “The Four Corner System (sijiao haoma 四角號碼).” ChinaKnowledge.de, 30 Oct. 2015, http://www.chinaknowledge.de/Literature/Script/sijiao.html.

Wang Yunwu 王雲五. “Haoma jianzifa” 號碼檢字法 [The Number Indexing Method]. Dongfang zazhi 東方雜誌 [The Eastern Miscellany], vol. 22, no. 12, 1925, pp. 82–98.

Wang Yunwu 王雲五. “Sijiao Haoma Jianzifa” 四角號碼檢字法 [The Four-Corner Indexing Method]. Dongfang zazhi 東方雜誌 [The Eastern Miscellany], vol. 23, no. 3, 1926, pp. 95–103.

Wang Yunwu 王雲五. Sijiao haoma jianzifa: fu jianzibiao 四角號碼檢字法：附檢字表 [The Four-Corner Indexing Method, with Index Tables]. Rev. ed., Taiwan Commercial Press, 1967.

Watanabe, Satosi. “Information Theoretical Analysis of Multivariate Correlation.” IBM Journal of Research and Development, vol. 4, no. 1, 1960, pp. 66–82, https://doi.org/10.1147/rd.41.0066.

Zar, Jerrold H. Biostatistical Analysis. 5th ed., Pearson Prentice Hall, 2010.